



Patroni Pending Restart Part I: A PostgreSQL Shared Memory Story



Jlluesma

Follow

12 min read · 1 day ago



Why Patroni tells you to restart replicas first when you raise `max_connections` and the primary first when you lower it. The answer is not in Patroni. It is in `ipci.c`.

A Tuesday afternoon change

The application team has been complaining for weeks: connection spikes during batch windows are hitting the `max_connections` ceiling. You run a Patroni-managed cluster so the fix looks trivial. You edit the dynamic configuration:

```
$ patronictl -c /etc/patroni/patroni.yml edit-config
```

```
postgresql:
  parameters:
    max_connections: 1000    # was 500
```

Patroni writes the new value to the DCS, every node picks it up, and each local PostgreSQL configuration is rewritten. Then you check the cluster:

```
$ patronictl list
```

```
+ Cluster: main (7212054711781907471) -----+-----+-----+-----+
| Member | Host       | Role   | State   | TL | Lag in MB | Pending restart |
+-----+-----+-----+-----+-----+-----+-----+
| pg1    | 10.0.0.11  | Leader | running | 12 |           | *               |
| pg2    | 10.0.0.12  | Replica | streaming | 12 | 0         | *               |
```

pg3	10.0.0.13	Replica	streaming	12	0	*
-----	-----------	---------	-----------	----	---	---

Every node is flagged **Pending restart**. Nothing is broken. Patroni is relaying what PostgreSQL itself reports in `pg_settings.pending_restart`: the new value is on disk, the running instances still operate with the old one, and this parameter cannot be applied by a reload. So far, so ordinary. The interesting part is the next question, the one every operator asks with the cursor hovering over the enter key:

Do I restart the whole cluster at once? The primary first? The replicas first? Does it even matter?

It matters. Get it wrong and one of your replicas will not come back the way you expect.

What the Patroni documentation says

Patroni treats six parameters specially. Five of them (`max_connections`, `max_worker_processes`, `max_wal_senders`, `max_prepared_transactions` and `max_locks_per_transaction`) are tied to shared-memory sizing and are the subject of this article. `wal_level` belongs to the same group for a different reason: it changes what information is written to the WAL stream, so **primary and standbys must also agree on it (Part II)**. Patroni forces these parameters into the DCS and rewrites every node's configuration accordingly. Until each node is restarted, though, the running cluster can temporarily contain different effective values, making the restart order critical. The documentation is blunt:

- **Increasing** one of these parameters → restart the **replicas first**, then the primary.
- **Decreasing** one of these parameters → restart the **primary first**, then the replicas.

Taken at face value, this reads like cargo cult. Why does the direction of the change flip the restart order? Why these parameters and not, say, `shared_buffers`, which also demands a restart? The Patroni docs cannot really answer, because the rule is not Patroni's rule. Patroni is quoting PostgreSQL: the hot standby chapter of the official documentation states the same constraint, almost word for word.

So the answer lives somewhere inside PostgreSQL, and this article is the walk from the `patronictl` prompt down to it. The route:

```

Patroni: "Pending restart"
      |
      v
Q1: why does PostgreSQL need a restart at all?
      |
      v
shared memory: computed once, then frozen
      |
      v
Q2: why would a STANDBY care about the
    PRIMARY's settings?
      |
      v
the WAL: XLOG_PARAMETER_CHANGE
      |
      v
recovery: CheckRequiredParameterValues()
      |
      v
A: the rolling restart order

```

Each stop answers one question and raises the next. The first sounds almost too basic to ask: why is a restart needed at all?

What the postmaster does before accepting a single connection

When PostgreSQL starts, the postmaster does something most databases postpone, amortize, or hide: it computes the size of **every** major internal structure it will ever need, adds them up, asks the operating system for one contiguous shared memory segment of exactly that size, and carves it up. Before the first backend forks. Before the first WAL record is replayed.

The entry point is `CreateSharedMemoryAndSemaphores()` in `src/backend/storage/ipc/ipci.c`, and the arithmetic lives right next to it. A few lines of it (PostgreSQL 17) are enough to see the pattern:

```

/* ipci.c, CalculateShmemSize(), abridged */
size = add_size(size, BufferShmemSize());      /* shared_buffers      */
size = add_size(size, LockShmemSize());        /* max_locks_per_xact  */
size = add_size(size, ProcGlobalShmemSize());  /* MaxBackends         */
size = add_size(size, XLOGShmemSize());        /* wal_buffers         */
size = add_size(size, CLOGShmemSize());        /* pg_xact SLRU        */
size = add_size(size, SUBTRANSShmemSize());   /* pg_subtrans SLRU    */
size = add_size(size, MultiXactShmemSize());   /* multixact SLRUs     */
size = add_size(size, ProcArrayShmemSize());   /* MaxBackends         */
size = add_size(size, ReplicationSlotsShmemSize()); /* max_replication_slots */
size = add_size(size, WalSndShmemSize());      /* max_wal_senders     */

```

```
...
return size;
/* ~30 more subsystems */
```

Every `*ShmemSize()` function is a pure function of configuration parameters and compile-time constants. **The size of PostgreSQL's shared memory is fully determined by the configuration at startup.** Change the configuration and the arithmetic changes; the only way to rebuild the structures is to rebuild the segment. `pending_restart` is the honest admission of that fact.

Much of that list is not the buffer pool. The transaction status machinery alone occupies several lines: `pg_xact` (commit/abort status), `pg_subtrans` (subtransaction parent links), MultiXact offsets and members, commit timestamps, and the `LISTEN/NOTIFY` queue are all SLRU ("simple LRU") page caches living in the same segment.

How the memory is actually obtained

On modern releases, PostgreSQL allocates its main shared-memory segment with a single anonymous `mmap()` call on POSIX systems

`src/backend/port/sysv_shmem.c`:

```
ptr = mmap(NULL, allocsize, PROT_READ | PROT_WRITE,
           PG_MMAP_FLAGS | mmap_flags, -1, 0);
/* where PG_MMAP_FLAGS = (MAP_SHARED|MAP_ANONYMOUS|MAP_HASSEMAPHORE) */
```

Once created, every backend inherits the same mapping after `fork()`, so shared structures remain at fixed addresses for the lifetime of the server.

Looking inside: `pg_shmem_allocations`

Since PostgreSQL 13, the segment we just watched being built is a view:

```
SELECT name, off, size, allocated_size
FROM   pg_shmem_allocations
ORDER BY allocated_size DESC
LIMIT 12;
```

name	off	size	allocated_size
Buffer Blocks	10267648	4294967296	4294967296
Buffer Descriptors	6710272	33554432	33554432
Buffer IO Condition Variables	4305235968	8388608	8388608
XLOG Ctl	54144	4208288	4208384

Xact	...	...	...
Subtrans	...	...	...
Lock Manager: LOCK hash	...	...	...
Lock Manager: PROCLock hash	...	...	...
Proc Array	...	...	...
KnownAssignedXids	...	...	...
Backend Status Array	...	...	...
Notify Queue	...	...	...

`off` is the fixed offset of each structure inside the segment assigned at startup, never moved. `allocated_size` includes padding. Sum it and you get the instance's total footprint:

```
SELECT pg_size_pretty(sum(allocated_size)) FROM pg_shmem_allocations;
```

`shared_buffers` dominates the output, and it is tempting to leave it at that: the buffer pool plus some change. But scroll past the first row and the segment turns out to be crowded: transaction bookkeeping, lock tables, per-backend state, replication machinery, all living at fixed offsets inside the same allocation.

The first question on the map is answered:

```
Patroni: "Pending restart"
      |
      v
✓ shared memory: computed once, then frozen <-- we are here
      |
      v
Q2: why would a STANDBY care about the
    PRIMARY's settings?
```

A restart is required because the configuration is the memory layout. But that says nothing about restart *order*.

The hidden multiplier: `MaxBackends`

`max_connections` never sizes anything by itself. What PostgreSQL uses everywhere is a value derived once, early in startup:

```
/* postinit.c, InitializeMaxBackends() */
MaxBackends = MaxConnections + autovacuum_max_workers +
```

```
max_worker_processes + max_wal_senders + NUM_SPECIAL_WORKER_PROCS;
```

(`NUM_SPECIAL_WORKER_PROCS` is 2: the autovacuum launcher and the slot sync worker.)

`max_connections`, `max_worker_processes` and `max_wal_senders` feed directly into `MaxBackends`. `max_locks_per_transaction` and `max_prepared_transactions` size shared-memory structures whose capacity is computed from it.

One example is the shared-memory lock table that stores PostgreSQL's heavyweight locks. Its size is computed by this macro in `src/backend/storage/lmgr/lock.c`:

```
#define NLOCKENTS() \
    mul_size(max_locks_per_xact, add_size(MaxBackends, max_prepared_xacts))
```

So on that Tuesday you did not merely raise a connection limit. You changed the value that PostgreSQL uses to size a whole family of shared-memory structures. Some grow directly from `MaxBackends`; others combine it with parameters such as `max_locks_per_transaction` or `max_prepared_transactions`. Together they change the result of `CalculateShmemSize()`, which is why PostgreSQL needs a new shared-memory segment after the restart.

Everything so far concerns a single instance minding its own business. Nothing yet explains why a *standby* should care what the primary's `max_connections` is. The answer is in the WAL.

The primary writes its configuration into the WAL

When one of those parameters changes on the **primary**, PostgreSQL does not treat it as a private matter. During startup, `XLogReportParameters()` compares each of the five (plus `wal_level`) against the control file, and on any difference does two things:

```
/* xlog.c, XLogReportParameters(), abridged */
if (MaxConnections != ControlFile->MaxConnections ||
    max_locks_per_xact != ControlFile->max_locks_per_xact || ...)
{
    recptr = XLogInsert(RM_XLOG_ID, XLOG_PARAMETER_CHANGE);
    ...
    ControlFile->MaxConnections = MaxConnections;
```

```
UpdateControlFile();
}
```

An `XLOG_PARAMETER_CHANGE` record enters the WAL stream, and `pg_control` is stamped with the new values. Every standby that replays the record repeats the second step in `xlog_redo()`: it copies the *primary's* values into its **own** control file, then immediately re-runs a compatibility check we will meet shortly. The stored values are visible on any node:

```
$ pg_controldata $PGDATA | grep -E "max_conn|worker|wal_senders|prepared|locks"
max_connections setting:          1000
max_worker_processes setting:      8
max_wal_senders setting:          10
max_prepared_xacts setting:        0
max_locks_per_xact setting:        64
```

On a standby, this is the standby's control file describing the **primary's** configuration. The standby permanently carries a record of how big the primary's transaction machinery is.

That is the second question on the map: why does a standby need to *know* that?

Hot standby: replaying someone else's transactions inside your own arrays

A hot standby does something that is easy to overlook: while it replays the primary's WAL, it also serves read-only queries. Those queries still need correct MVCC snapshots, so the standby must maintain its own in-memory view of the transactions currently running on the primary.

Everything the standby knows about the primary's transaction state has to be reconstructed locally from WAL records and stored in its own shared-memory structures. One of those structures is `KnownAssignedXids`:

```
#define PROCARRAY_MAXPROCS (MaxBackends + max_prepared_xacts)
```

```
/*
 * Ideally we'd only create this structure if we were actually doing hot
 * standby in the current run, but we don't know that yet at the time
 * shared memory is being set up.
 */
```

```
#define TOTAL_MAX_CACHED_SUBXIDS \
((PGPROC_MAX_CACHED_SUBXIDS + 1) * PROCARRAY_MAXPROCS)
```

The code above is not creating the array; it is computing how large it must be. `KnownAssignedXids` list of every transaction ID the standby currently believes is running on the primary. Every read-only snapshot on the standby is built from it, and its capacity is fixed at startup from the standby's own `MaxBackends`.

The second mirror is the lock table. When the primary takes an `AccessExclusiveLock` (`DROP TABLE`, `TRUNCATE`, many `ALTER TABLE` s), the lock is WAL-logged, and the startup process acquires the same lock locally (`StandbyAcquireAccessExclusiveLock()`, `standby.c`) so that read-only queries cannot scan a table being destroyed underneath them. Those locks land in the standby's own `NLOCKENTS()`-sized hash.

Put the two halves of the article side by side and the incompatibility appears in its natural habitat:

PRIMARY (max_connections = 1000)		STANDBY (max_connections = 500)
-----		-----
up to ~1015 concurrent xids	---WAL-->	<code>KnownAssignedXids</code> sized for ~515
<code>AccessExclusiveLocks</code> , up to	---WAL-->	lock hash sized for
64 x (1015 + prepared)		64 x (515 + prepared) entries
		
THE STREAM CAN EXCEED THE CONTAINER		

If the standby's arrays are smaller than the primary's transaction-generating capacity, the WAL stream can legally describe a state the standby is physically unable to represent. Eight hundred concurrent transactions is a valid workload on that primary and an impossible object for a `KnownAssignedXids` array sized for five hundred connections. There is no graceful degradation: dropping an xid would silently corrupt every snapshot on the standby; dropping a replicated lock would let a query read a table mid- `TRUNCATE` .

PostgreSQL refuses to gamble. `CheckRequiredParameterValues()` (in `xlog.c`) compares the standby's local GUCs against the primary's values from the control file:


```

if (ArchiveRecoveryRequested && EnableHotStandby)
{
    /* We ignore autovacuum_max_workers when we make this test. */
    RecoveryRequiresIntParameter("max_connections",
                                MaxConnections,
                                ControlFile->MaxConnections);
    ...
}

```

The enforced rule is per-parameter and directional: **standby value $\geq$ primary value**. The check runs at standby startup *and* re-fires inside `xlog_redo()` whenever an `XLOG_PARAMETER_CHANGE` record arrives, which is how a standby that was happily streaming can hit the wall while running.

A misconception worth killing explicitly

None of this is about “memory layouts matching” between nodes. Each instance has its own virtual address space; an address from the primary would be meaningless noise on the standby. Nothing of the kind crosses the wire:

travels in the WAL:	never travels:
-----	-----
transaction IDs	pointers
subxid relationships	offsets
lock tags (dbOid, relOid)	shared memory addresses
running-xacts snapshots	memory layouts
slot metadata	

The compatibility requirement exists because the standby must *materialize* that semantic state inside its own local, fixed-capacity structures. The structures do not need identical addresses or layouts. They need sufficient **capacity**.

The map is nearly complete:

```

✓ shared memory: computed once, then frozen
    |
    v
✓ the WAL: XLOG_PARAMETER_CHANGE
    |
    v
✓ recovery: CheckRequiredParameterValues()
    |
    v
A: the rolling restart order

```

What happens when you restart in the wrong order

Back to Tuesday. You raised `max_connections` from 500 to 1000, everything shows `Pending restart`, and you restart the primary first.

It comes up at 1000. `XLogReportParameters()` stamps the control file and writes `XLOG_PARAMETER_CHANGE`. Seconds later both replicas, still running at 500, replay that record. What happens next depends *when* the mismatch is detected.

PostgreSQL 14 and newer: deliberate paralysis, then a hard stop. If the mismatched record arrives while hot standby is up and serving queries the standby does not die:

```
WARNING: hot standby is not possible because of insufficient parameter settings
DETAIL:  max_connections = 500 is a lower setting than on the primary server,
        where its value was 1000.
LOG:     recovery has paused
DETAIL:  If recovery is unpaused, the server will shut down.
HINT:    You can then restart the server after making the necessary
        configuration changes.
```

Existing read-only sessions keep working, but WAL replay is frozen at the `XLOG_PARAMETER_CHANGE` record, with the startup process parked in a condition-variable loop. Request promotion and it refuses (`promotion is not possible because of insufficient parameter settings`). Unpause without fixing the GUC and it deliberately dies with `recovery aborted because of insufficient parameter settings`. That same `FATAL` fires immediately when the check trips at startup, before hot standby is active so restarting the paused replica without a config fix leaves the node crash-looping until the configuration is fixed.

From Patroni's point of view the paused member is `running`, state `streaming` ... with replication lag growing without bound. Arguably this failure mode is operationally *more* dangerous than the old versions crash-loop, because nothing is red. Only the lag column give it away, while the primary quietly accumulates a WAL-retention problem.

Deriving the rolling restart rule from first principles

The Patroni documentation stops being cargo cult and becomes a corollary. The invariant PostgreSQL enforces, at startup and at every

`XLOG_PARAMETER_CHANGE`:

```
at every instant: standby capacity >= primary capacity
```

Walk both directions through it.

Increasing (500 → 1000): replicas first. Each replica comes up at 1000 while the primary still runs at 500. Invariant holds: $1000 \geq 500$. When all replicas are done, restart the primary; its `XLOG_PARAMETER_CHANGE` replays into arrays that are already big enough.

Decreasing (1000 → 500): primary first. The primary comes up at 500 and WAL-logs the decrease. The replicas, still at 1000, remain compatible because PostgreSQL only requires **standby $\geq$ primary**. They can then be restarted to 500 one by one.

INCREASE (replicas **first**)

DECREASE (**primary first**)

The asymmetry has a single root cause, and it is the whole article in one sentence: **only the standby has to fit someone else's workload into its own fixed arrays.** The primary never replays the standby's transactions; the standby always replays the primary's. Capacity must flow downhill, and the restart order is just the schedule that keeps it flowing downhill at every intermediate step.

Closing the loop

The map we set out with is complete, and every arrow on it has a name from the source tree:

```
Patroni: "Pending restart"                                (pg_settings.pending_restart)
    |
    v
✓ shared memory: computed once, then frozen                (CalculateShmemSize, ipci.c)
    |                                                        (MaxBackends, postinit.c)
    v
✓ the WAL: XLOG_PARAMETER_CHANGE                          (XLogReportParameters, xlog.c)
    |
    v
✓ recovery: standby >= primary                            (CheckRequiredParameterValues
    |                                                        (KnownAssignedXids, NLOCKENTS
    v
✓ the rolling restart order                               (the only schedule that keeps
    |                                                        the invariant true throughou
```

Every arrow is architecture, not policy. Shared memory is computed once and frozen because pointer stability, lock-free snapshot scans, and fail-at-boot semantics were chosen over runtime elasticity. Hot standby must mirror the primary's transaction and lock state inside arrays frozen the same way, so capacity acquires a direction.

Patroni never was the story. It flagged the restart, it will orchestrate it in the right order. The rolling restart rule you skimmed past in two sets of documentation is not an operational recommendation at all. It is PostgreSQL's oldest allocation decision projected through the WAL onto your change calendar.

Pending restart is simply PostgreSQL acknowledging that its shared-memory layout is now out of sync with its configuration.

One parameter remains: `wal_level`. It follows the same operational rule, but for an entirely different architectural reason. That story begins in the WAL and belongs in Part II.

. . .

References

- PostgreSQL documentation — [Hot Standby: Administrator's Overview](#) (the canonical statement of the parameter rule and restart order)
- PostgreSQL documentation — [pg_shmem allocations](#)
- PostgreSQL documentation — [Shared Memory and Semaphores](#)
- Patroni documentation — [PostgreSQL parameters controlled by Patroni](#)
- Source: [src/backend/storage/ipc/ipci.c](#) — `CalculateShmemSize()`, `CreateSharedMemoryAndSemaphores()`
- Source: [src/backend/utils/init/postinit.c](#) — `InitializeMaxBackends()`
- Source: [src/backend/storage/ipc/shmem.c](#) — `ShmemInitStruct()`, the `ShmemIndex`
- Source: [src/backend/storage/lmgr/proc.c](#) — PGPROC / ProcGlobal initialization
- Source: [src/backend/storage/lmgr/lock.c](#) — `NLOCKENTS()`, lock hash sizing
- Source: [src/backend/storage/ipc/procarray.c](#) — `PROCARRAY_MAXPROCS`, `TOTAL_MAX_CACHED_SUBXIDS`, `KnownAssignedXids`

- Source: `src/backend/storage/ipc/standby.c` — `StandbyAcquireAccessExclusiveLock()`
- Source: `src/backend/access/transam/xlog.c` — `XLogReportParameters()`, `CheckRequiredParameterValues()`, `xlog_redo()`
- Source: `src/backend/access/transam/xlogrecovery.c` — `RecoveryRequiresIntParameter()` (the pause/FATAL logic)
- Commit `15251c0a` (PostgreSQL 14) — "Insufficient parameter settings now pause recovery instead of erroring out"
- Commit `941697c3` (PostgreSQL 14) — removal of `PGXACT`, dense arrays in `ProcGlobal`

Postgresql

Postgres

Database



Written by Jlluesma

5 followers · 2 following

Follow



